```
function bubbleUp(pairs,
index=|pairs|-1)                                    #1
  parentIndex ← index
                                  #2
  while parentIndex >
0                                                     #3
    currentIndex ← parentIndex
    parentIndex ←
getParentIndex(parentIndex)                                  #4
    if pairs[parentIndex].priority <
pairs[currentIndex].priority then        #5
      swap(pairs, currentIndex,
parentIndex)                                #6
    else
                #7
      break
```

#1 We explicitly pass the array with all pairs and the index (by default the last element) as arguments. In this context, `|A|` means the size of an array `A`.

#2 Start from the element at the index passed as argument (by default the last element of A)

#3 Check if the current element is already at the root of the heap: if so, we are finished

#4 Compute the index of the parent of current element in the heap. The formula can vary with the type of implementation. For a heap with branching factor `D` and array's indices starting at zero, it's `(parentIndex-1)/D`

#5 If the parent has lesser priority than the current element p…

#6 … then swaps parent and children

#7 Otherwise, we are sure that heap's properties are reinstated, and we can exit the loop and return